

Programación Evolutiva - Práctica 2

Óscar Durán Narganes, Izan de Vega López

Índice

Índice	2
Algoritmos inventados	3
Cruce múnico	3
Mutación piedrapapeltijera	14
Capturas	20
Tabla comparativa fitness	22
Conclusiones	22
Reparto de tareas	23
Créditos adicionales	23

Algoritmos inventados

Cruce múnico

El algoritmo de *cruce múnico*, llamado en honor al ritual de combate gladiatorio romano conocido como *munes*, es un método de cruce de permutaciones inventado por nosotros, que pretende simular batallas entre dos individuos de la población para crear descendencia.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'								
2'								

Dados dos individuos con la codificación genética

1: (1, 2, 7, 4, 6, 3, 5, 8)

2: (2, 1, 3, 4, 8, 6, 5, 7)

Y un factor de armadura A = 2 (Empleado para ajustar ciertos cálculos).

La descendencia se calcula de la forma siguiente:

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'								
2'								

Primero, elegimos el primer elemento de cada uno de los individuos progenitores, que en este caso son 1 y 2. Este será el valor de velocidad.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'								
2'								

A continuación, elegimos el elemento izquierdo de cada uno como valor de ataque, y su valor derecho como valor de armadura...

A = 2

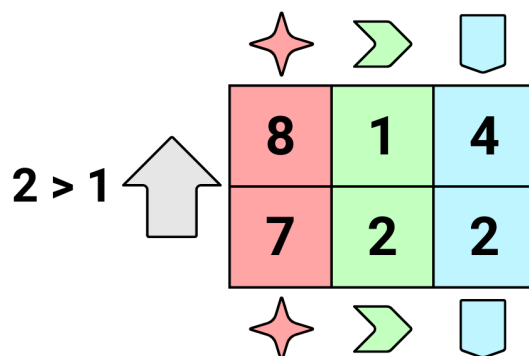
1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7

***A**

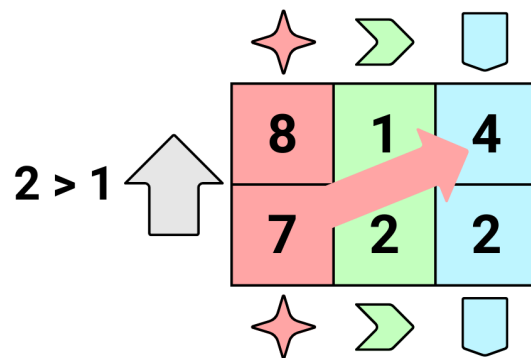
8	1	4						
7	2	2						
2'								

...Y creamos un gladiador a partir de cada uno de ellos, multiplicando el valor de armadura por A.

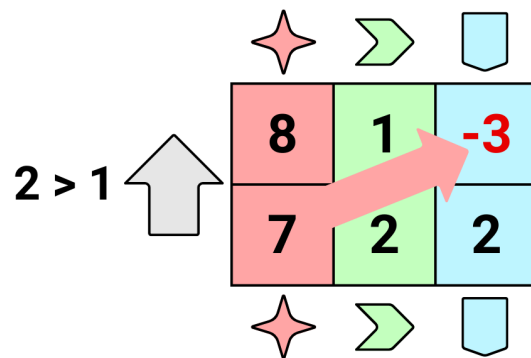
(La implementación del factor de armadura fue realizada pues nos percatamos de que la velocidad podía ser decisiva a la hora de elegir elementos comparada con los otros atributos. Este puede modificarse según se quiera)



Una vez hayamos creado ambos gladiadores, estos empiezan a luchar. Estos se golpean por turnos, empezando por el que mayor valor de velocidad tenga.



En cada turno, el gladiador atacante usará su valor de ataque para disminuir la armadura del defensor.



Cuando la armadura de cualquier gladiador disminuya a 0 o menor, este gladiador perderá el combate, mientras que el otro será coronado como ganador.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2							
2'	1							

Los gladiadores ganadores serán introducidos en el primer individuo, mientras que los perdedores serán introducidos en el segundo.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2							
2'	1							

Continuamos en el algoritmo, decidiendo el siguiente elemento a heredar.

$A = 2$

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2							
2'	1							

Sin embargo, vemos que ambos elementos ya están contenidos en los nuevos individuos, por lo que existe la posibilidad de romper la permutación.

$A = 2$

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1						
2'	1	2						

En este caso, el combate será evitado, y los elementos serán introducidos en el individuo en el que no estén.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8			
2'	1	2	3	4	6			

Continuamos rellenando los nuevos individuos hasta llegar a otro elemento de interés.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8			
2'	1	2	3	4	6			

En este cruce, nos fijamos que tanto el 3 como el 6 se hallan en el mismo individuo, por lo que tampoco podemos realizar un combate.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8	3	6	
2'	1	2	3	4	6			

Por lo tanto, seguimos el ejemplo anterior, e introducimos ambos elementos en el individuo que no los contenga ya.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8	3	6	
2'	1	2	3	4	6			

En este caso, el puntero de la descendencia se desplazará, permitiendo rellenar los individuos de forma desacompañada.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8	3	6	5
2'	1	2	3	4	6	5		

Continuamos rellenando los individuos.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8	3	6	5
2'	1	2	3	4	6	5		

Para el último elemento, nos percatamos de que el primer descendiente está lleno, por lo que no puede aceptar más elementos.

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8	3	6	5
2'	1	2	3	4	6	5		

No obstante, esto no debe considerarse un caso especial. Como el primer individuo ya incluye a todos los elementos, únicamente debemos aplicar los principios anteriores...

A = 2

1	1	2	7	4	6	3	5	8
2	2	1	3	4	8	6	5	7



1'	2	1	7	4	8	3	6	5
2'	1	2	3	4	6	5	8	7

...E introducir los elementos repetidos en el individuo en el que no estén ya.

Resultado

1'	2	1	7	4	8	3	6	5
2'	1	2	3	4	6	5	8	7

Esto nos devuelve dos nuevos individuos, con las codificaciones:

1: (2, 1, 7, 4, 8, 3, 6, 5)

2: (1, 2, 3, 4, 6, 5, 8, 7)

Mutación piedrapapeltijera

El algoritmo de *mutación piedrapapeltijera* es un algoritmo de mutación de permutaciones basado en el juego de piedra, papel o tijera, simulando un torneo entre los elementos de cada individuo para modificarlo.

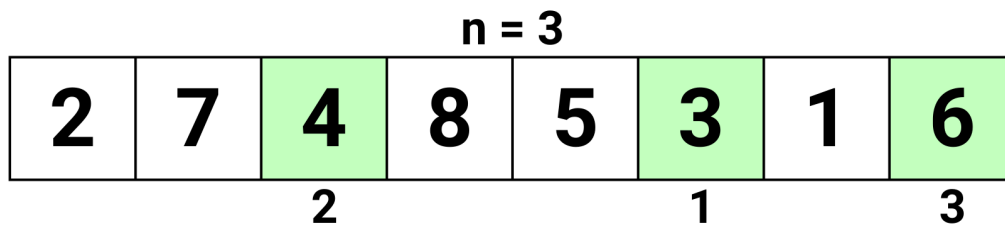
n = 3

2	7	4	8	5	3	1	6
----------	----------	----------	----------	----------	----------	----------	----------

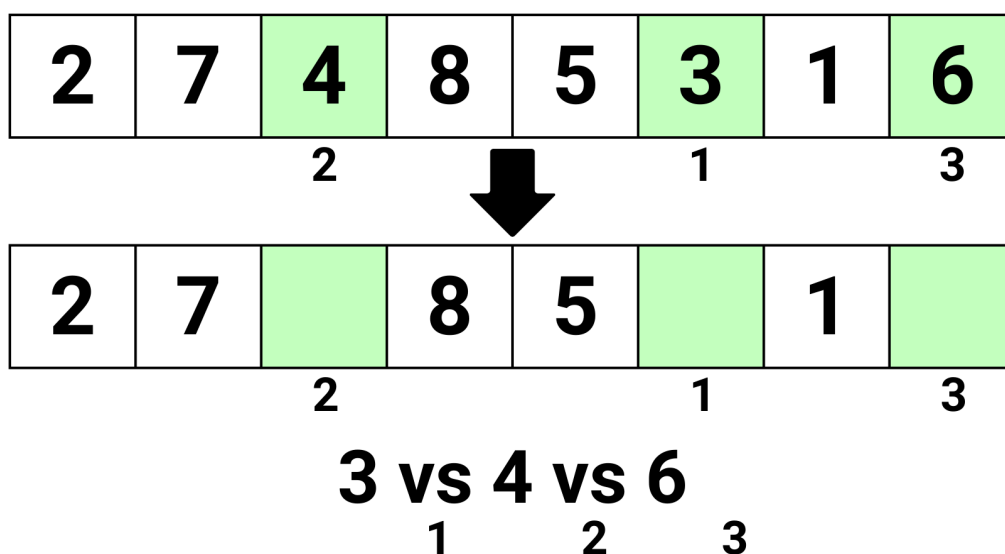
Dados un individuo con la codificación genética

(2, 7, 4, 8, 5, 3, 1, 6)

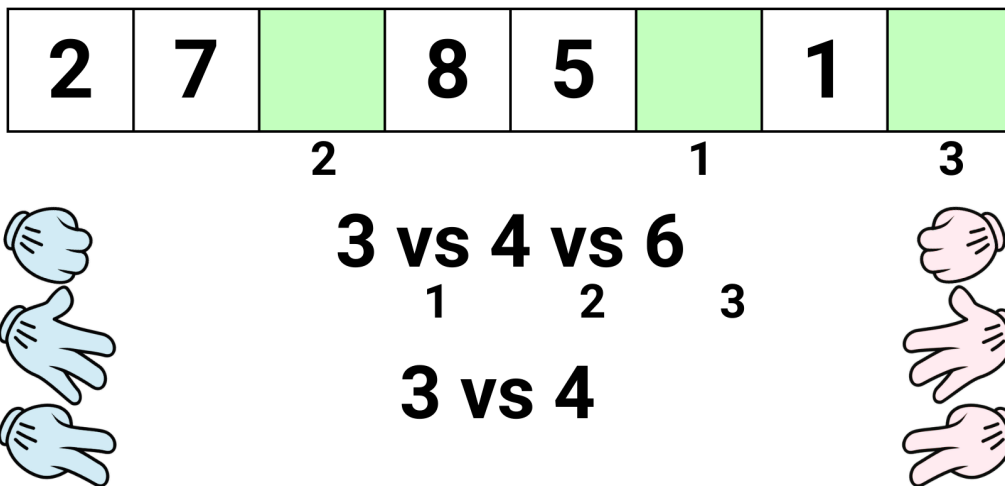
Para el que queremos mutar $n = 3$ posiciones, realizamos los siguientes pasos:



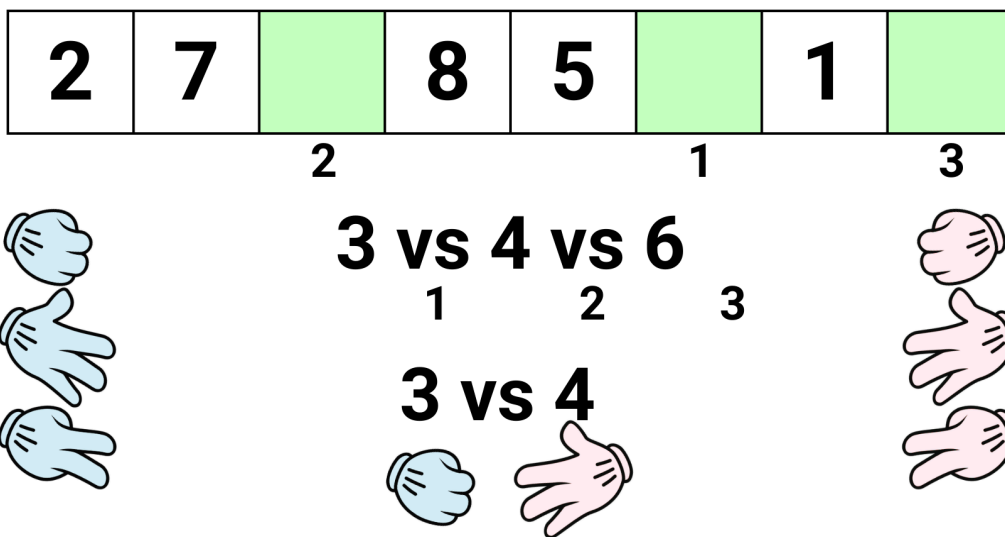
Elegimos tres posiciones distintas, enumeradas en orden de selección. En este caso, estas son las posiciones 5, 2 y 7 (con índice 0).



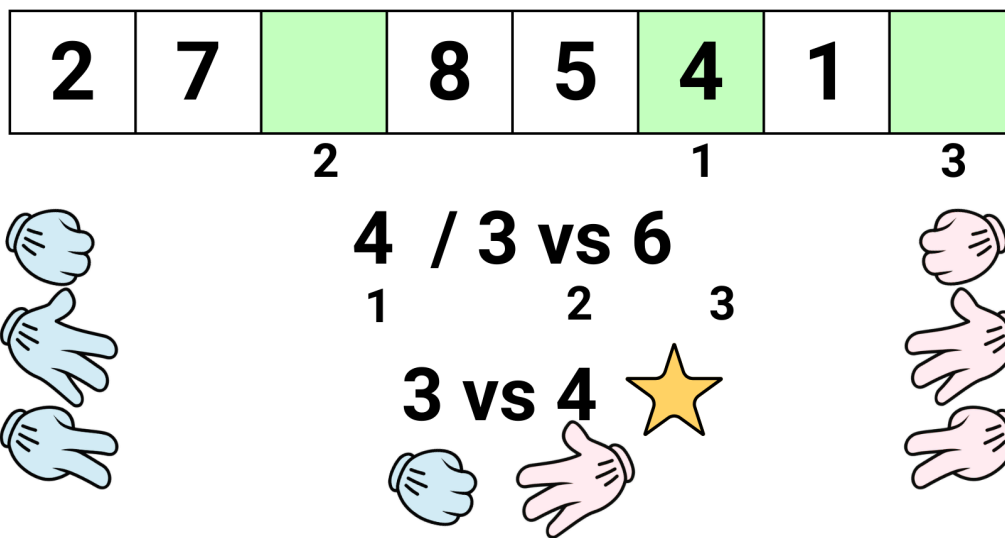
Extraemos los elementos de las posiciones dadas, y realizamos $n-1$ batallas para decidir la posición final de cada uno. Cada elemento competirá contra el siguiente por una posición, y en el caso de que pierda o empate, competirá por la posición siguiente.



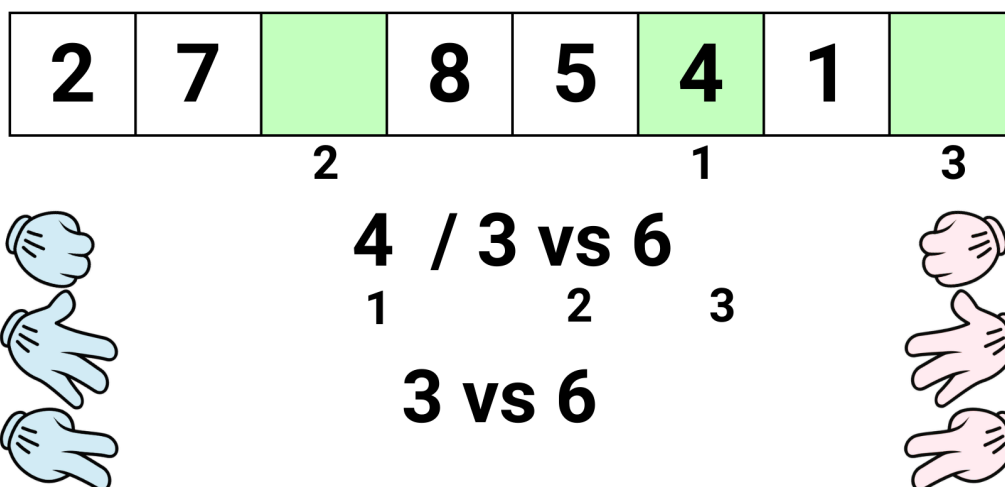
Estas batallas, naturalmente, se realizarán mediante piedra, papel o tijera. En la primera batalla compiten el 3 y el 4 por el primer puesto.



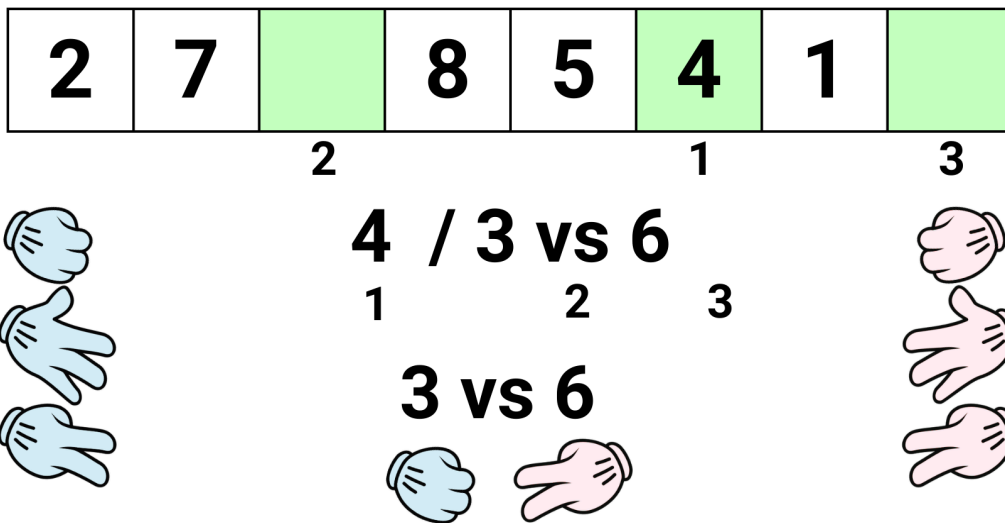
Eligiendo aleatoriamente, el 3 saca piedra, mientras que el 4 saca papel.



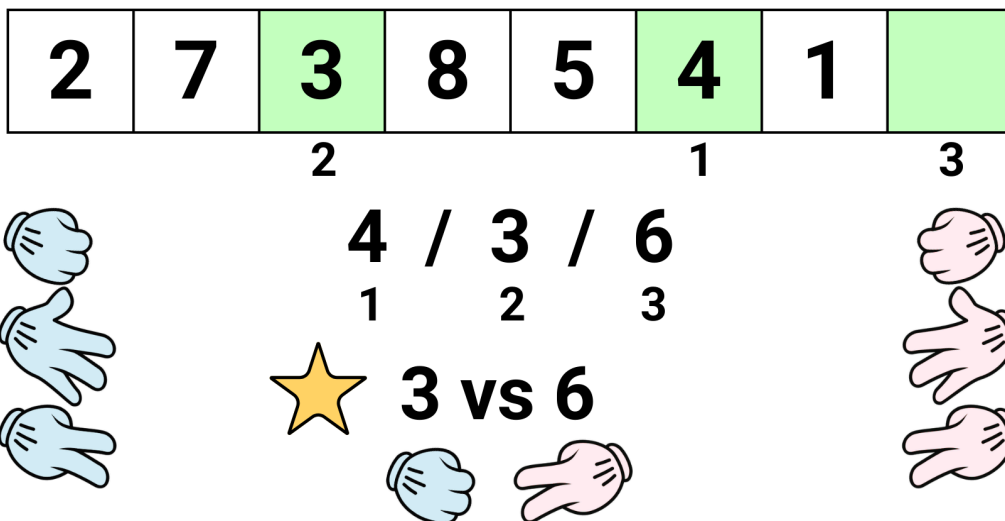
Esto significa que el 4 gana y se queda con la primera posición, mientras que el 3 va a la batalla siguiente.



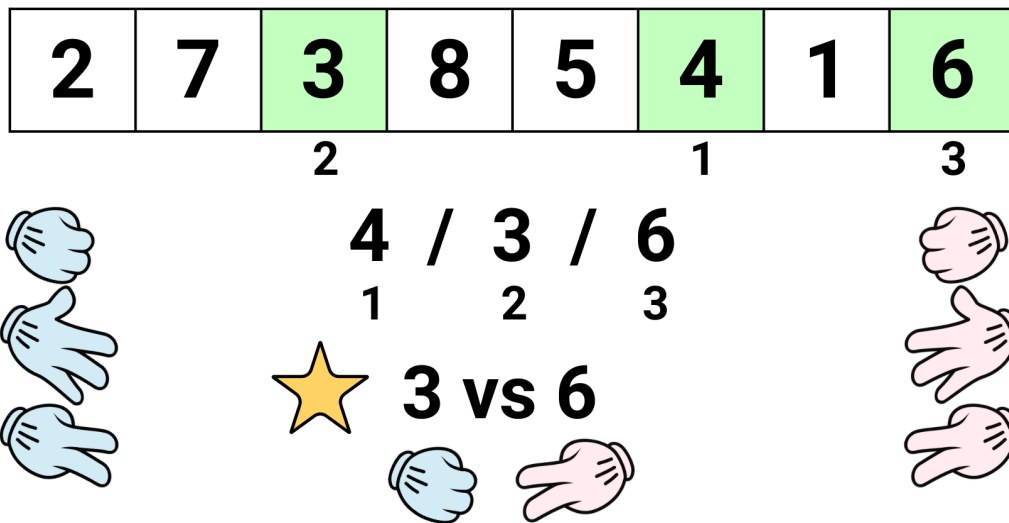
Ahora, el 3 y el 6 luchan por la segunda posición.



El 3 saca piedra, y el 6 saca tijera.



Por lo tanto, el 3 gana, y se queda con la segunda posición.



Como no quedan más concursantes, el 6 se queda con la última posición.

Resultado

2	7	3	8	5	4	1	6
---	---	---	---	---	---	---	---

Esto nos da un individuo mutado con la codificación genética siguiente:

(2, 7, 3, 8, 5, 4, 1, 6)

Capturas

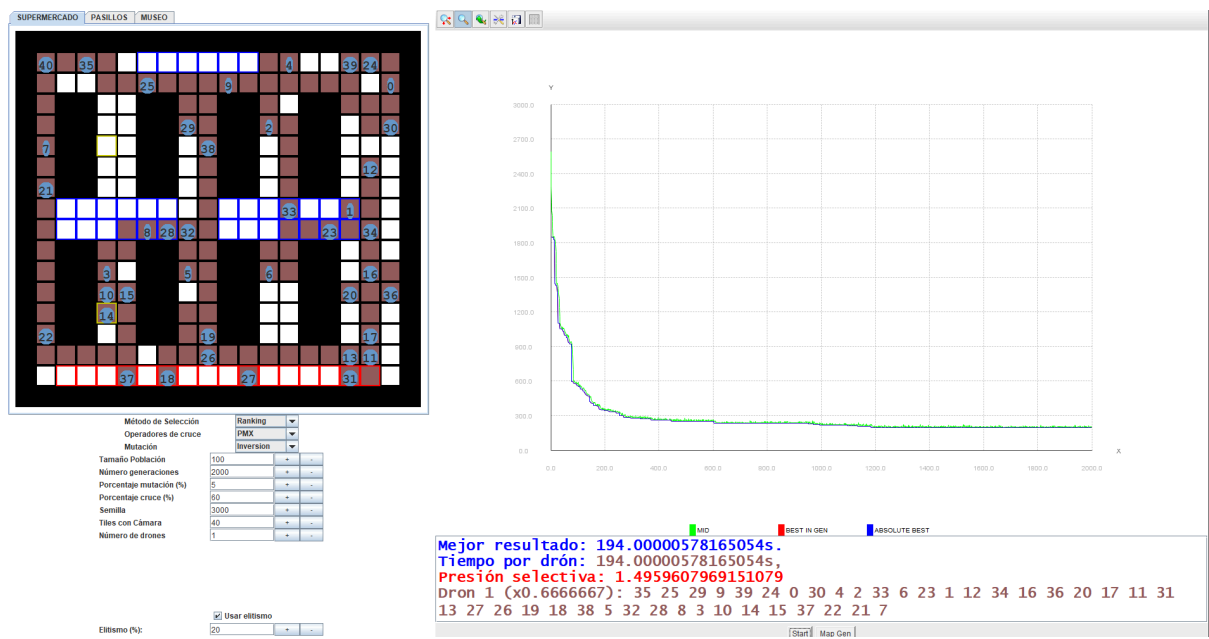
1 drón

Cruce: PMX

Mutación: Piedra Papel Tijera

Selección: Ranking

Elitismo: 20%



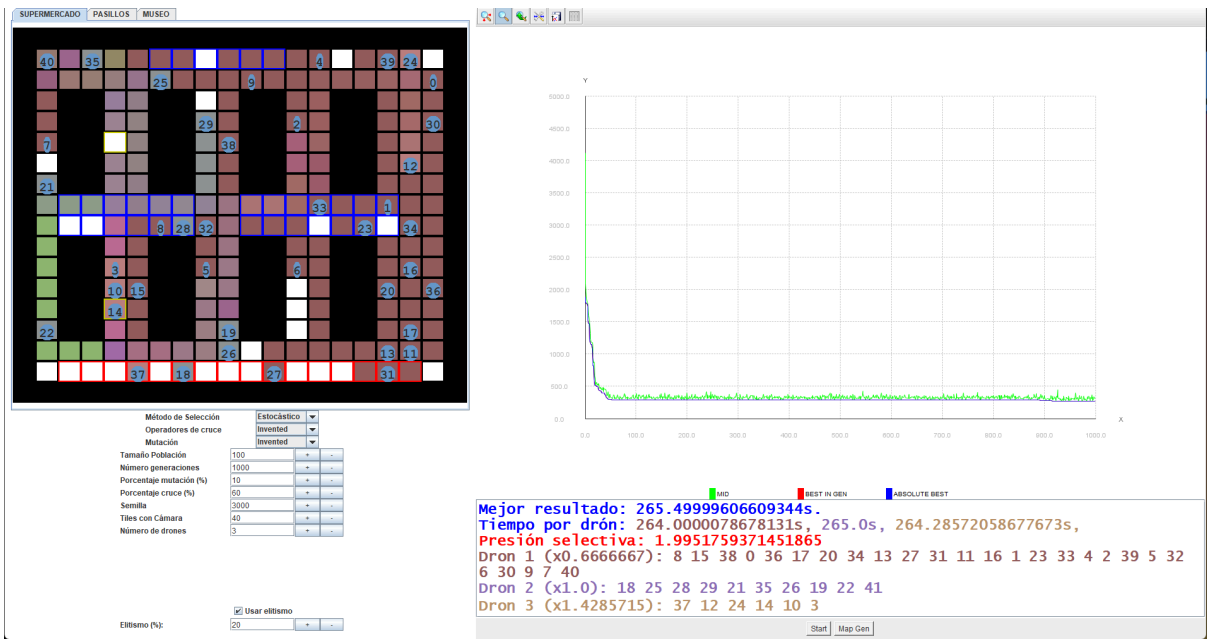
3 drones

Cruce: Múnico

Mutación: Piedra Papel Tijera

Selección: Estocástico

Elitismo: 20%



5 drones

Cruce: Múnico

Mutación: Piedra Papel Tijera

Selección: Ranking

Elitismo: 20%

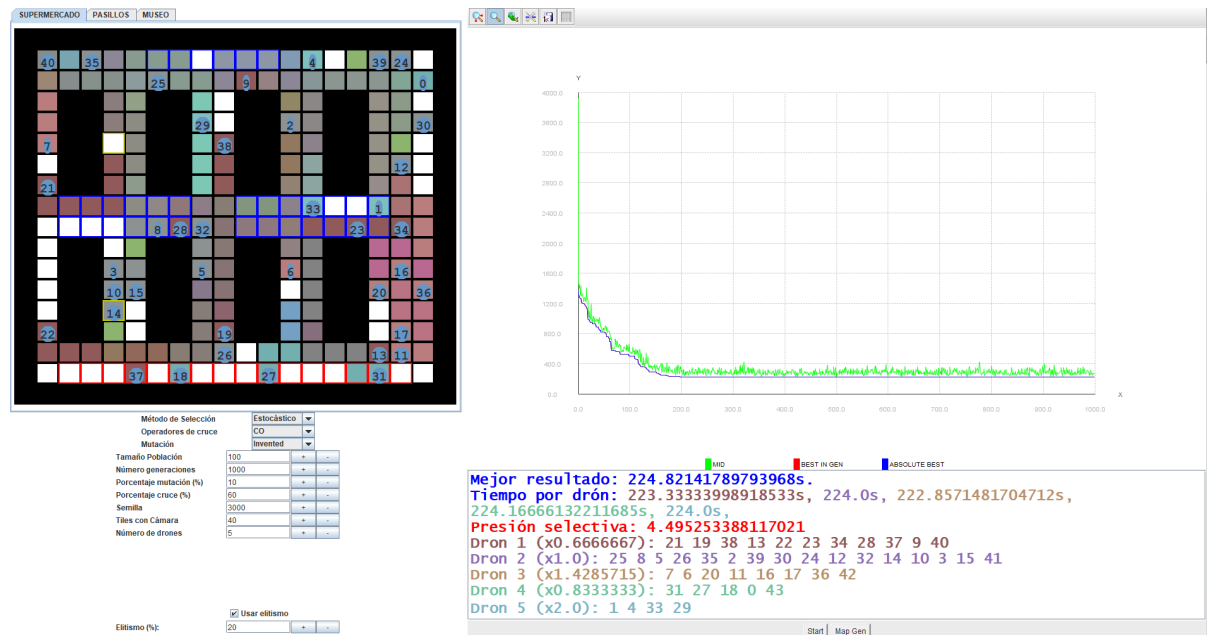


Tabla comparativa fitness

1 dron	3 drones	5 drones
194	265	224.82

Conclusiones

Hemos observado que aumenta el tiempo al dividir el trabajo entre más drones. Suponemos que es porque al no poder pasar por todas las casillas es más fácil pasar por casillas que ya tienen cámaras, o tener que esquivarlas haciendo un recorrido por casillas de menor peso. Todo esto sumado a que el primer dron es rápido y su multiplicador reduce considerablemente el valor de fitness que obtenga.

Los drones rápidos (el 1 y el 4), reciben con mayor frecuencia la tarea de ir a los puntos con mayor peso, como se puede ver en las capturas.

Los drones pesados (el 3 y el 5), reciben un menor número de casillas que cubrir como se puede ver claramente en la imagen con 3 drones

Reparto de tareas

- Apartado visual de la práctica con Java Swing y MathPlot → Óscar Durán
- Algoritmo de pathfinding A* → Óscar Durán e Izan de Vega
- Algoritmos de cruce → Izan de Vega
- Algoritmos de mutación → Izan de Vega
- Adaptación de métodos de selección → Izan de Vega
- Algoritmo de selección por ranking → Óscar Durán
- Algoritmo de cruce inventado (cruce múnico) → Óscar Durán
- Algoritmo de mutación inventado (mutación piedrapapeltijera) → Óscar Durán
- Unión apartado visual con resultados de los algoritmos → Izan de Vega
- Memoria → Óscar Durán e Izan de Vega

Créditos adicionales

- [Gráficos de piedra, papel o tijera por OpenClipart.](#)